

Diseño de un Agency OS para una pequeña agencia de branding y marketing (2026)

Resumen ejecutivo

Los softwares más exitosos para agencias (Productive, Scoro, Teamwork, Bonsai, Monday, ClickUp, Asana, Notion, Airtable, HubSpot, Pipedrive, etc.) convergen en un patrón claro: una plataforma centralizada que integra CRM, proyectos, tiempo, finanzas y documentación, con vistas de reporting orientadas a rentabilidad y utilización. Las agencias modernas operan con un stack ligero pero bien orquestado, donde un "operating layer" coordina la relación entre herramientas comerciales (CRM, email, ads) y herramientas internas (PM, documentación, BI).
[1][2][3][4][5][6]

Para una agencia pequeña (5–15 personas) que quiere un Agency OS interno con potencial de evolucionar a SaaS, el camino más eficiente es:

- MVP: un monolito web tipo Next.js con base de datos Postgres (Supabase o Railway), autenticación simple (Clerk/Auth.js), y foco en cuatro módulos: CRM básico, proyectos/tareas, tiempo + facturación ligera, y documentación/SOPs.[7][4][^6]
- Arquitectura: monolito bien estructurado (clean modules) + espacio reservado para un futuro "internal tools" layer (Retool/Metabase) sobre los datos, evitando microservicios prematuros.[8][9][^10]
- Desarrollo con IA: usar un IDE AI-first (Cursor o Windsurf) apoyado en modelos Claude Sonnet/Opus para diseño y refactorizaciones profundas, más herramientas de app-building (v0 para UI, Bolt/Lovable solo para prototipos).[11][12][^13]
- Web corporativa: proyecto unificado (web + app) desplegado en Vercel, con marketing pages y app protegida por auth, dejando Shopify para e-commerce futuro si aplica.[6][14][^15]

El informe profundiza en mercado, operación de agencias, definición del MVP, arquitectura tecnológica, stack de IA, opciones de web corporativa, UX, riesgos y un roadmap recomendado a 12 meses.

Parte 1 — Análisis de mercado de software para agencias

Panorama general

Guías recientes de "agency management software" coinciden en que las herramientas líderes para agencias combinan gestión de proyectos, tiempo, recursos y facturación en un solo sistema, con integraciones hacia CRM y contabilidad. Productive, Scoro y Teamwork se posicionan explícitamente como plataformas "end-to-end" para agencias y servicios

profesionales, frente a herramientas horizontales como Monday, Asana, ClickUp y Notion.^[16]
^[17]^[18]^[1]

Además, muchos pequeños estudios creativos montan su propio "Agency OS" sobre Notion y Airtable, usando plantillas avanzadas que conectan CRM, proyectos, finanzas, portales de cliente y SOPs en un mismo workspace. Estas soluciones suelen ser suficientes hasta ~10–20 personas, pero empiezan a mostrar límites en permisos, rendimiento y reporting financiero avanzado.^[19]^[20]^[^21]

Tabla comparativa: foco principal y módulos

Herramienta	Problema principal que resuelve	Lo que hace especialmente bien	Lo que odian sus usuarios	Módulos clave (para agencias)
HubSpot	CRM + marketing automation	CRM integrado con email, páginas, automatizaciones; buen reporting. ^[22] ^[23]	Coste creciente con contactos y hubs, cierta complejidad en workflows avanzados. ^[24] ^[25]	CRM, marketing, tickets, reporting.
Pipedrive	CRM de ventas simple	Pipeline visual drag-and-drop, UX muy intuitiva, onboarding rápido. ^[22] ^[26]	Marketing limitado, menos apto para procesos complejos; reporting menos profundo que Salesforce. ^[24] ^[27]	CRM, pipeline, actividades, email básico.
Salesforce	CRM enterprise altamente configurable	Customización extrema, módulos sectoriales, reporting muy potente. ^[22] ^[25]	Curva de aprendizaje alta, dependencia de administradores, costes de implantación elevados. ^[24] ^[25]	CRM, ventas, servicio, marketing, analítica avanzada.
Monday	Work OS visual	Tableros muy configurables, automatizaciones tipo "recetas", dashboards y vistas múltiples. ^[28] ^[7]	Puede volverse complejo, funciones financieras y de tiempo menos profundas que herramientas específicas. ^[7] ^[29]	Proyectos, tareas, automatizaciones, paneles, algo de CRM.
ClickUp	All-in-one productividad	Gran profundidad de tareas, jerarquía, docs, time tracking y automations en un solo producto. ^[28] ^[7]	UI percibida como saturada y abrumadora, curva de aprendizaje fuerte. ^[28] ^[7]	Proyectos, docs, time tracking, objetivos, chat.
Asana	Task manager estructurado	Reglas, dependencias, Timeline, Portfolios y Goals muy sólidos para servicios recurrentes. ^[28] ^[7]	Menos enfoque en finanzas, facturación y cliente externo; podría requerir otras apps complementarias. ^[7] ^[30]	Proyectos, tareas, objetivos, workload.
Linear	Issue tracking para producto	Fluidez y velocidad, excelente para equipos de producto/engineering. ^[^30]	Enfoque estrecho en issues; menos adecuado para CRM/finanzas/cliente. ^[^30]	Issues, roadmap, ciclos.

Herramienta	Problema principal que resuelve	Lo que hace especialmente bien	Lo que odian sus usuarios	Módulos clave (para agencias)
Notion	Workspace conectado	Bases de datos, wiki, portales de cliente y Agency OS relacional en un solo espacio. ^{[31][19][^20]}	Requiere diseño de sistema y mantenimiento; tareas y notificaciones menos robustas que PM dedicado. ^{[7][29]}	Documentación, CRM ligero, proyectos, SOPs.
Airtable	Base de datos tipo spreadsheet	Modelado de datos flexible, vistas, automatizaciones, buen como backend ligero. ^{[32][3]}	Escalar permisos y lógica compleja es costoso; UX menos amigable como herramienta diaria para todo el equipo. ^[^32]	Tablas relacionales, automatizaciones, integraciones.
Teamwork	PM para client work	Enfoque en agencias: time tracking, clientes, facturación y portals. ^{[33][5][^34]}	UI percibida como menos moderna, puede ser demasiado para equipos muy pequeños. ^{[33][35]}	Proyectos, tiempo, billing, client portal, recursos.
Productive	End-to-end agency OS	Visibilidad de utilización, rentabilidad y forecasting en tiempo real; todo en una sola plataforma. ^{[4][36]}	Setup más pesado que Notion/ClickUp; coste superior a herramientas genéricas; requiere adopción disciplinada. ^{[37][38]}	CRM, proyectos, recursos, tiempo, reporting financiero, facturación.
Scoro	PSA para agencias B2B	Proyectos, tiempo, retainer billing, reporting financiero y reciente capa de AI (ELI). ^{[39][40]}	Configuración compleja, percibido como "enterprise" para algunos equipos pequeños. ^{[41][42]}	CRM, proyectos, time billing, finanzas, reporting, AI insights.
Bonsai	Freelancers y micro-agencias	Propuestas, contratos, facturas y tracking de proyectos simple en un solo lugar. ^{[43][44]}	Limitado para equipos >10 y operaciones complejas; PM menos profundo que herramientas dedicadas. ^[^44]	CRM ligero, contratos, facturas, proyectos básicos.
Retool	Internal tools y dashboards	Construcción rápida de dashboards, CRUD interno y admin panels con datos existentes. ^{[8][45]}	Enfoque en internal tools, no en UX pública; coste por usuario puede crecer. ^{[45][46]}	UI interna, dashboards, workflows, conexiones a DB/APIs.
Metabase	BI y analítica self-service	Tableros y exploración de datos sobre stack existente; ideal para KPIs de agencia. ^{[10][47]}	No es PM ni CRM; requiere stack de datos mínimamente ordenado. ^[^10]	Dashboards, consultas, exploración de datos.
Make	Integraciones y no-code automation	Orquesta herramientas (Airtable, Notion, CRM, DAM) con flujos visuales complejos. ^{[32][3][^48]}	Flujos mal diseñados se vuelven frágiles; coste por ejecuciones si no se optimiza. ^[^32]	Automations, ETL ligero, orquestación de stack.
n8n	Automations auto-hosteables	Alternativa open-source a Make/Zapier con más control y menor coste recurrente. ^{[3][49]}	Requiere más operación técnica; UI menos pulida. ^[^3]	Automations, integraciones, workflows.

Qué funciones son imprescindibles para una agencia pequeña

Del análisis de reseñas y guías de stack para agencias de 2025-2026 emergen varios mínimos imprescindibles:[¹⁷][¹][^{^6}]

1. CRM ligero pero claro

- Contactos, empresas, deals, pipeline visual, estados y valor estimado.[²²][²⁶]
- Vista rápida de historial de interacciones (emails, reuniones, notas).[^{^32}]

2. Gestión de proyectos orientada a client work

- Proyectos con tareas, deadlines, responsables, estados.[³³][⁵]
- Vistas por cliente y por equipo; plantillas para tipos de proyectos recurrentes (branding, web, campañas).[¹⁷][⁷]

3. Time tracking y rentabilidad básica

- Registro de horas por tarea/proyecto, distinción billable vs no billable.[⁴¹][³³]
- Reportes sencillos de utilización, margen por proyecto y por cliente.[³⁹][³⁶]

4. Facturación y contratos al menos integrables

- Generación de facturas desde horas o presupuestos.[³⁹][⁴]
- Registro de propuestas/contratos aceptados y facturas emitidas (aunque el cobro se haga en otra herramienta como Stripe/contabilidad).[⁴⁴][¹⁵]

5. Documentación y SOPs centralizados

- Wiki de procesos, plantillas, guías de brand, estrategias.[²⁰][²¹]
- Capacidad de vincular SOPs a tipos de proyectos y checklists.[⁵⁰][²⁰]

6. Dashboards operativos simples

- Visión semanal de carga de trabajo por persona, proyectos críticos, pipeline de ventas y facturación mensual.[³⁶][⁶]

Funciones innecesarias o peligrosas para una agencia pequeña

Basado en opiniones de usuarios y recomendaciones de consultores, varias funciones suelen ser sobrepeso para una agencia de <20 personas:[⁴¹][⁶][^{^17}]

- Módulos de ERP complejo (inventarios, compras avanzadas) que no aplican a servicios.
- Workflows excesivamente sofisticados de aprobación en 5 niveles; el coste organizativo supera el beneficio.[^{^42}]
- Configuraciones de permisos hipergranulares al estilo enterprise cuando la cultura es de transparencia; añaden fricción.[^{^33}]
- Automatizaciones sin disciplina de diseño: Make/n8n con decenas de escenarios no versionados que se rompen y nadie recuerda qué hacían.[⁴⁸][³²]
- Reporting ultra detallado sin data hygiene: intentar replicar un stack de data warehouse cuando la base de datos aún no está estable.[⁵¹][¹⁰]

Patrones de UX y módulos recurrentes

Las herramientas más adoptadas comparten ciertos patrones de UX:[²⁸][5][⁷]

- Barra lateral con **secciones principales**: CRM, Proyectos, Tiempo, Finanzas, Documentación.
- Vistas de lista/tabla con filtros + vistas de tablero (Kanban) para pipelines y tareas.[²⁸][7]
- Encabezados con KPIs clave (deals abiertos, horas esta semana, utilización, margen).[³⁶][33]
- Navegación centrada en **entidades**: Cliente → Proyectos → Tareas → Horas → Facturas.

Los módulos que aparecen constantemente en software exitoso para agencias incluyen:[⁵²][1][¹⁷]

- CRM/pipeline, proyectos/tareas, time tracking, recursos/capacidad, presupuestos, facturación, reporting, documentación interna, client portals.

Parte 2 — Cómo operan las agencias modernas

Estructura operativa típica

Estudios sobre tech stack de agencias de marketing y creative ops muestran estructuras operativas convergentes:[⁵³][3][⁵⁴]

- **Front office**: dirección, estrategia, cuentas/CSM, ventas/BD.
- **Delivery**: branding, diseño, copy, digital, desarrollo web.
- **Ops & finanzas**: operaciones, PMO ligero, finanzas, a veces RevOps.

Las agencias eficientes utilizan un stack claro por capa: CRM/marketing automation (HubSpot/Pipedrive), PM/time tracking (Asana, Monday, ClickUp, Teamwork, Productive), documentación (Notion), BI ligero (Metabase/Looker Studio) y orquestación con Make/n8n.[⁵⁵][32][⁶]

Organización interna y gestión de clientes

Buenas prácticas comunes:[³][56][⁵³]

- **Un único registro de cliente** que vincula CRM, proyectos y reporting; se evita duplicar datos en múltiples sistemas.
- **Account owners** definidos por cliente, con vistas de salud de cuentas, proyectos activos y pipeline de upsell.[⁵⁶]
- **Uso de portales de cliente** (Teamwork, Notion, AgencyPro) para compartir estados, entregables y documentación.[¹⁹][52][³³]

Gestión de proyectos y reporting

Agencias eficientes:[⁶¹33][[^]36]

- Trabajan con **plantillas de proyectos** por tipo de servicio: rebrand, web, campaña, retainer, etc.
- Aplican **time tracking** razonable (no obsesivo) para alimentar KPIs de utilización y margen por proyecto.[⁴¹33]
- Reportan a dirección al menos mensualmente: utilización, margen por cliente, facturación vs forecast, y pipeline.[⁵¹36]

Automizaciones y sistemas internos

Tendencias:[³²3][[^]48]

- Uso de **Make/Zapier/n8n** para conectar CRM ↔ PM ↔ email ↔ DAM, eliminando reentrada manual.
- Automatización de generación de creatividades y reporting con AI y herramientas como Abyssale, Medialake, etc.[⁵⁷48]
- Internal tools con Retool u otras plataformas low-code para vistas ad-hoc sobre datos internos.[⁵⁸8]

Prácticas de eficiencia más habituales

Según guías de tech stack y estudios de agencias:[⁵³3][[^]6]

- **Consolidar herramientas:** reducir el número de sistemas y usar uno o dos hubs fuertes (por ejemplo, PM/time + documentación).[⁵⁹6]
- **Estandarizar procesos** y SOPs conectados a plantillas de proyectos para evitar variabilidad en calidad.[²⁰50]
- **Medir CLV, margen y utilización** además de métricas superficiales como clics o tiempo estimado.[⁶⁰53]

Sistemas que terminan abandonando y sustituciones típicas

Patrones observados:[⁶¹59][[^]41]

- Agencias que intentan usar **solo Notion como PM** acaban añadiendo Asana/Monday/ClickUp para ejecución de tareas.[⁷¹20]
- Herramientas de PM genéricas (Asana/Monday) son sustituidas por **Productive/Teamwork/Scoro** cuando la agencia pasa a obsesionarse por rentabilidad y capacidad.[¹⁸40][[^]1]
- Varios CRMs ligeros son reemplazados por **HubSpot** cuando se necesita marketing automation serio; Pipedrive se mantiene donde el foco es puramente ventas.[²⁶62]

Errores recurrentes

Errores que se repiten:[⁵³][32][[^]6]

- **Comprar demasiadas herramientas** sin proceso ni adopción, generando "tool sprawl" y datos fragmentados.
- **No invertir en documentación y SOPs**, lo que hace que cada proyecto se gestione de forma distinta.[⁵⁰][20]
- **No medir utilización y rentabilidad** hasta que ya hay problemas financieros.
- **Automatizar sin governance**, con scripts y escenarios frágiles que nadie mantiene.[⁴⁸][32]

Parte 3 — Definición del MVP ideal

Principios de priorización

Para una agencia pequeña con ambición de producto futuro, el MVP debe:[²⁰][6]

- Resolver dolores internos claros desde el primer mes.
- Evitar intentar competir de inmediato con Productive/Scoro, pero inspirarse en sus conceptos.
- Diseñar desde el principio un modelo de datos limpio que pueda soportar evolución a SaaS.

Día 1 — Debe existir

1. Core CRM de agencia

- Entidades: Empresa, Contacto, Deal, Servicio.[²²][26]
- Pipeline simple con etapas (Lead, Discovery, Proposal, Negotiation, Won/Lost).
- Campos clave: valor estimado, probabilidad, fecha cierre, origen, owner.

2. Gestión básica de proyectos y tareas

- Proyecto ligado a un Deal ganado y a un Cliente.[⁵][33]
- Tareas con estado, responsable, fechas, estimación de horas.[[^]28]
- Vistas: tablero Kanban por estado, lista por responsable, calendario.

3. Time tracking mínimo y reporting esencial

- Registro de horas por tarea (billable/no billable).[⁴][33]
- Reporte simple: horas por proyecto, por persona y por cliente; utilización aproximada.

4. Documentación interna compacta

- Wiki con páginas para brandbook, planes tipo, estrategias, SOPs y plantillas.[²¹][20]
- Capacidad de vincular una SOP/plantilla a tipo de proyecto.

5. Roles y permisos básicos + autenticación

- Roles: Admin, Project Lead, Member, Viewer.

- Autenticación con proveedor externo (Clerk/Auth.js) y soporte para acceso cliente futuro.

Fase 2 — Después de validar uso interno

1. Finanzas y facturación ligera

- Presupuestos ligados a proyectos; estados (draft, accepted, invoiced).
- Facturas simples generadas desde horas o hitos; export a PDF/contabilidad.^[39]^[4]

2. Capacidad y planificación de recursos

- Vista de carga por persona (horas/semana, proyectos activos).^[33]^[36]
- Indicación de sobrecarga y huecos; forecast de disponibilidad sencilla.

3. Dashboards operativos

- KPIs: utilización por equipo, margen básico por proyecto, facturación mensual vs objetivo, pipeline de ventas.^[51]^[36]

4. Automatizaciones internas low-code

- Webhooks/API básicos para conectar con Make/n8n y crear flujos: cuando se gana un Deal → crear proyecto y tareas base; cuando se emite factura → enviar a contabilidad.^[32]^[48]

Fase 3 — Funcionalidades avanzadas y potencial SaaS

1. Client portal

- Vista para clientes con proyectos, tareas relevantes, estados, entregables y facturas.^[19]^[33]

2. Módulo de análisis avanzado de rentabilidad

- Cálculo más fino de overhead, coste hora, márgenes por cliente y servicio siguiendo modelos como el de Productive.^[36]^[39]

3. Automatizaciones de AI y asistentes internos

- Asistente para generar propuestas, resumir proyectos, detectar riesgos y sugerir mejoras de procesos.^[63]^[4]

4. Multi-tenant y features de SaaS comercial

- Separar claramente tenants, billing por workspace, planes, límites de usuarios y datos.

Parte 4 — Arquitectura tecnológica

Evaluación de tecnologías front-end y back-end

Next.js vs React "puro"

- **Next.js**
 - Ventajas: SSR/SSG automático, routing integrado, API routes, integración nativa con Vercel, excelente DX para apps SaaS.^{[14][15]}
 - Desventajas: ligera complejidad añadida; requiere entender data-fetching híbrido (server/client components).
- **React "puro"**
 - Ventajas: simplicidad conceptual si solo se construye SPA; flexible sobre cualquier backend.^[^64]
 - Desventajas: más trabajo para routing, SSR, SEO y server-side; menos alineado con despliegues en Vercel.

Para un Agency OS con potencial de SaaS, Next.js es la elección pragmática: acelera SEO, routing, SSR para dashboards y facilita hosting en Vercel.^{[14][6]}

Supabase vs Firebase vs Postgres "a medida" + Prisma

- **Supabase (Postgres gestionado + auth + storage)**
 - Ventajas: Postgres estándar, auth, storage y APIs ya listas; gran integración con Next.js y ecosistema moderno.^[^6]
 - Desventajas: dependencias en su plataforma; algunas features son más nuevas que equivalentes de Firebase.
- **Firebase (Firestore)**
 - Ventajas: tiempo real sencillo, SDKs muy maduros, buen soporte para auth y hosting móvil/web.^[^15]
 - Desventajas: modelo de datos no relacional y consultas más limitadas; peor ajuste para reporting relacional como el de una agencia (clientes, proyectos, horas, facturas).
- **Postgres gestionado (Railway/Render) + Prisma**
 - Ventajas: control total del esquema relacional, independencia de vendor; Prisma aporta DX fuerte para tipos y migraciones.^[^6]
 - Desventajas: requiere montar auth, storage y APIs manualmente; algo más de trabajo inicial.

Dado que la agencia quiere reporting financiero y relacional (clientes ↔ proyectos ↔ horas ↔ facturas), una base relacional es preferible. Para velocidad y simplicidad, Supabase o Railway+Postgres+Prisma son mejores opciones que Firebase.^{[10][6]}

Autenticación: Clerk vs Auth.js

- Clerk
 - Ventajas: componentes listos para login, equipos, multi-tenant, sesión; excelente DX con Next.js.^[^6]
 - Desventajas: coste recurrente según usuarios y features.
- Auth.js (antes NextAuth)
 - Ventajas: open-source, flexible, permite integración con múltiples proveedores OAuth y credenciales.^[^64]
 - Desventajas: más trabajo manual para UI, invitaciones, equipos y manejo de sesiones complejas.

Para un pequeño equipo con posible crecimiento SaaS, Clerk reduce fricción en auth multi-tenant y roles, aunque Auth.js es viable si se prioriza coste y control.

Hosting y despliegue: Vercel vs Railway vs Render

- Vercel
 - Ventajas: hosting ideal para Next.js, despliegues automáticos, preview envs, edge functions.^{[15][14]}
 - Desventajas: modelo de precios basado en tráfico y funciones; backend pesado puede no encajar si se usa intensivamente.
- Railway
 - Ventajas: hosting sencillo para Postgres y servicios Node; buena para pequeña infraestructura.^[^6]
 - Desventajas: menos especializado en front; topes de uso gratuitos limitados.
- Render
 - Ventajas: similar a Railway, soporte para contenedores, cron jobs, etc.^[^6]
 - Desventajas: ligeramente más compleja de configurar; latencias a considerar.

Un patrón común para SaaS de tamaño pequeño es Next.js en Vercel + Postgres/Prisma en Railway/Render o Supabase.^{[10][6]}

Facilidad para trabajar con IA

Para integrar AI (asistente interno, generación de propuestas, automatizaciones inteligentes):

- Next.js ofrece APIs server-side para integrar LLMs (Claude, OpenAI, etc.) con buen control de tokens y costos.^{[12][65]}
- Postgres/Prisma/Supabase permiten guardar contexto semiestructurado (prompts, logs, insights) con facilidad.^[^6]
- Vercel y otras plataformas ya exponen patrones de "AI routes" y edge functions para inferencia ligera.^{[14][15]}

Arquitectura recomendada

Para el contexto concreto (pequeña agencia, equipo técnico reducido, fuerte uso interno y potencial evolución a SaaS):

- **Front-end:** Next.js (App Router) + React, TypeScript, Tailwind/Chakra según preferencia.
- **Back-end:** Postgres (Supabase o Railway) + Prisma como ORM.
- **Auth:** Clerk para rapidez (o Auth.js si se prioriza control).
- **Hosting:** Vercel para front; DB en Supabase/Railway.
- **Analytics/BI:** Metabase conectado a la base para dashboards avanzados de utilización y rentabilidad cuando sea necesario.^{[47][10]}
- **Automatización:** Make/n8n para integraciones con email/CRM externos y orquestación.

Esta arquitectura favorece velocidad de desarrollo, simplicidad y una ruta clara de escalabilidad: se puede mantener como monolito modular durante bastante tiempo antes de extraer servicios específicos.

Parte 5 — Desarrollo con IA

Estado actual de las herramientas de coding asistido (2026)

Comparativas recientes posicionan a Cursor, Windsurf y Claude Code como los tres pilares de IDEs/entornos AI-first para desarrollo serio, mientras que herramientas como Lovable, Bolt y v0 se centran más en generación de apps/UI con menos control.^{[66][65][^12]}

- **Claude Code**
 - Integración con MCP, buen manejo de codebases grandes, refactorización masiva, generación de tests y documentación; fuerte rendimiento en benchmarks tipo SWE-Bench.^{[67][12]}
- **Cursor**
 - Extensión de VSCode-like editor, contexto amplio, comandos de refactor, chat, buen para trabajo diario de desarrollo.^{[68][69]}
- **Windsurf**
 - Entorno orientado a multi-agent y "flow-state" coding; fuerte en automatización y file-tree handling para equipos.^{[65][67]}
- **Lovable / Bolt / v0**
 - Lovable: generación full-stack a partir de prompts, rápido para MVPs, menos control sobre el código generado y más lock-in.^{[70][13]}
 - Bolt.new: editor cloud AI-first con buen control para devs, cloud-only y más orientado a prototipos rápidos.^[^13]
 - v0: especializado en UI/React + Tailwind, ideal para generar componentes y pantallas de interfaz con calidad.^{[71][13]}

Rankings de "vibe coding" y AI dev tools coinciden en que los desarrolladores más productivos usan combinaciones, no una herramienta única: IDE AI-first (Cursor/Windsurf) + LLM de alto nivel para diseño/refactor (Claude Opus/Sonnet) + generadores de UI (v0).^{[72][73][^66]}

Stack ideal de IA para este proyecto

Objetivos: minimizar tokens, mantener calidad de código, soportar proyecto grande y equipo pequeño.

- **Modelo base:** Claude Sonnet para tareas diarias (menos coste, buen equilibrio) y Claude Opus para diseño de arquitectura, grandes refactorizaciones y decisiones complicadas.^{[67][12]}
- **Entorno de desarrollo:** Cursor o Windsurf como IDE principal; ambos ofrecen buen manejo de repos grandes, comandos por archivo/carpeta y flujos de trabajo productivos.^{[68][65]}
- **Generación de UI:** v0 para diseñar layouts de dashboards y componentes React con Tailwind de forma rápida, importando luego el código al repo principal.^{[71][13]}
- **Herramientas de app building (Lovable/Bolt):** usarlas solo para experimentar variantes de features o prototipos aislados; evitar depender de ellas para el core del producto por temas de lock-in y calidad de arquitectura.^{[70][13]}

Cuál desperdicia menos tokens y genera mejor código

Comparaciones apuntan a:^{[66][12][^65]}

- **Claude Code + Sonnet:** buena gestión de contexto, tendencia a explicaciones densas pero se puede controlar el estilo y reducir tokens ajustando prompts y settings.
- **Cursor/Windsurf:** como wrappers/IDEs, su consumo depende del modelo subyacente; con Sonnet/Opus o GPT-5, el coste se controla vía configuración (max tokens, frecuencia de llamadas) y hábitos (usar comandos breves).
- **Lovable/Bolt/v0:** pueden consumir muchos tokens en reconstrucciones repetidas del mismo app al iterar; algunos casos reportan millones de tokens en MVPs simples.^{[74][75]}

Por calidad de código y mantenibilidad, los artículos especializados recomiendan usar IDEs AI-first (Cursor/Windsurf) con LLM robusto y reservar los builders (Lovable, Bolt, v0) para piezas concretas o prototipos, no para el core de productos que deben escalar.^{[76][73][^11]}

Flujo de trabajo de desarrolladores productivos

Los workflows descritos en comparativas y guías de "AI dev stack" incluyen:^{[65][67][^68]}

1. Spec-driven development

- Escribir primero un PRD/feature spec claro (a menudo generado y refinado con IA).^[^77]
- Pedir al modelo que proponga arquitectura de módulos, entidades y contratos.

2. Scaffolding asistido

- Usar Cursor/Windsurf para generar boilerplate de Next.js, estructuras de carpetas, modelos Prisma y endpoints básicos.

3. Desarrollo iterativo con comandos

- Trabajar archivo a archivo, usando comandos de "improve", "add tests", "refactor" sobre código existente.

4. Review humano + AI

- Hacer code review manual, apoyado por análisis de IA para detectar smells, duplicidades y mejorar DX.

5. Documentación y SOPs técnicas con IA

- Generar y mantener documentación arquitectónica y guías de contribución con ayuda del LLM.^[^67]

6. Tests y refactorizaciones masivas

- Utilizar Claude Code o similares para generar suites de tests y refactorizar módulos enteros, con supervisión humana.^[^12]

Workflow propuesto para construir el Agency OS con IA

1. Fase de diseño (Semana 0–2)

- Redactar PRD detallado de Agency OS (módulos, entidades, flujos) en lenguaje claro.
- Usar Claude Opus/Sonnet para proponer arquitectura de datos (esquema Postgres) y estructura de módulos Next.js.

2. Scaffolding técnico (Semana 2–3)

- Con Cursor/Windsurf, generar proyecto Next.js + Prisma + auth (Clerk/Auth.js) + layout base.
- Crear migraciones iniciales para entidades: Cliente, Contacto, Deal, Proyecto, Tarea, TimeEntry, Invoice.

3. Construcción de módulos MVP (Semana 3–8)

- CRM: listar/crear/editar empresas, contactos y deals; pipelines y filtros.
- Proyectos/tareas: vistas por cliente, tablero, calendario; flujos de creación ligados a deals.
- Time tracking: formularios simples de registro de horas, vistas por persona/proyecto.
- Wiki: estructura de páginas y relación con tipos de proyecto.
- IA: primer asistente para generar resúmenes de proyectos y propuestas desde datos CRM.

Cada módulo se desarrolla con iteraciones cortas dirigidas por specs, usando IA para boilerplate y refactor.

4. Integraciones iniciales (Semana 8–10)

- Webhooks/API REST/GraphQL para Make/n8n.
- Opcional: integración simple con HubSpot/Pipedrive para sincronizar deals.^{[3][32]}

5. Refactor y preparación de Fase 2 (Semana 10–12)

- Pasar Claude Code sobre el repo para identificar mejoras de arquitectura, duplicidades, tests faltantes.
- Consolidar módulo de reporting inicial y preparar estructura para facturación.

Parte 6 — Web corporativa y software

Opciones y criterios

Seis arquitecturas comunes para agencias:[⁷⁸][14][^6]

- A) Web corporativa separada + aplicación separada.
- B) Web corporativa y aplicación dentro del mismo proyecto.
- C) Shopify + aplicación externa.
- D) Shopify + aplicación integrada.
- E) Headless Shopify.
- F) CMS moderno + aplicación.

Criterios comparados: coste, SEO, escalabilidad, mantenimiento, velocidad, complejidad.[¹⁵][6]

Comparación sintetizada

- A) Web corporativa separada + app separada
 - Coste: más alto (dos proyectos y despliegues).[⁶]
 - SEO: bueno si la web se optimiza; app separada no afecta.[¹⁵]
 - Escalabilidad: cada proyecto crece independiente; más trabajo de coordinación.
 - Mantenimiento: doble stack; más carga.
- B) Web corporativa y app en el mismo proyecto (Next.js)
 - Coste: menor; un solo repo y pipeline.[¹⁴]
 - SEO: excelente si se usan páginas estáticas/SSR para marketing; app protegida por auth.[¹⁵]
 - Escalabilidad: sencilla mientras se mantenga modularidad.
 - Mantenimiento: una base de código, rutas separadas para web y app.
- C/D/E) Combinaciones con Shopify
 - Relevantes sobre todo si la agencia vende productos/plantillas/digital goods directamente a escala.[¹⁴]
 - Para un Agency OS interno con potencial SaaS, Shopify no aporta mucho, salvo que se quiera monetizar plantillas o cursos.
 - Headless aporta flexibilidad pero suma complejidad innecesaria para este caso.[¹⁵]
- F) CMS moderno + app

- CMS (por ejemplo, headless como Sanity/Contentful o WordPress) puede alimentar la web corporativa, mientras la app vive en otro proyecto.^[15]
- Aporta a marketing equipos no técnicos, pero complica el stack; quizás excesivo para un equipo pequeño.

Para una agencia pequeña centrada en servicios y un OS interno, la opción B) (web y app dentro del mismo proyecto Next.js) es la más eficiente: minimiza costes, mantiene SEO, acelera desarrollo y reduce complejidad.^{[14][6]}

Parte 7 — Experiencia de usuario

Patrones UX en CRMs modernos y dashboards SaaS

Guías de CRM UX destacan patrones:^{[22][64]}

- **Layout de 3 zonas:** navegación lateral, zona de filtros y barra superior de acciones, contenido principal con listas/tableros.
- **Vistas salvables** (saved views) por usuario: por ejemplo, "Mis deals esta semana", "Proyectos activos por cliente", "Horas de hoy".^{[33][36]}
- Dashboards con **cards de KPIs** arriba y tablas/gráficos abajo.

En herramientas de productividad (ClickUp, Asana, Monday, Notion Agency OS), los layouts que mejor funcionan son:^{[7][28][19]}

- Vistas de tablero para pipelines y tareas.
- Listas y tablas para reporting y vistas densas.
- Calendar/timeline para campañas y proyectos largos.

Errores frecuentes de UX

Según comparativas de PM software y reseñas:^{[28][7][33]}

- **UI sobrecargada:** demasiadas columnas, filtros y campos obligatorios; usuarios se pierden y evitan usar la herramienta.
- **Navegación inconsistente:** cambiar de paradigma (por cliente, por proyecto, por vista) sin coherencia.
- **Falta de enfoque en casos de uso diarios:** no está claro qué deben hacer los usuarios al entrar cada día.

Cómo reducir fricción y evitar interfaces saturadas

Buenas prácticas:^{[7][20][6]}

- **Diseñar home dashboards por rol:** para PMs, para creativos, para dirección; con 3–5 KPIs y listas accionables.

- Mantener formularios con **campos mínimos**, dejando campos avanzados opcionales o en secciones colapsables.[²⁸]
- Usar **progressive disclosure**: mostrar más detalles solo cuando se expande una fila o se entra en un detalle.
- Ofrecer **comandos rápidos** (search omnibox) para navegar por clientes, proyectos y tareas sin recorrer el árbol.[³³]

Cómo diseñar una herramienta que realmente se use

Hilos en comunidades de agencias y guías de adopción repiten:[⁷][³³][¹⁴]

- La herramienta debe **reducir trabajo**, no aumentarlo: por ejemplo, auto-rellenar datos, templates y atajos.
- La adopción mejora cuando se conecta a **beneficios visibles**: menos caos en proyectos, menos horas no facturadas, más claridad en carga de trabajo.[³⁶]
- Empezar con **pocas vistas bien diseñadas** y expandir tras observar uso real.

Parte 8 — Riesgos

Errores comunes al construir software interno

1. **Scope creep**: intentar replicar Productive/Scoro desde el día 1, añadiendo todo tipo de módulos.[⁴⁰][¹]
2. **Tool sprawl**: construir un sistema que simplemente añade otra herramienta a un stack ya saturado, sin consolidar.[⁶¹]
3. **Falta de ownership**: nadie es responsable de producto; el OS se degrada con cambios ad hoc.

Errores de arquitectura

- Microservicios prematuros: separar servicios antes de tener necesidades reales, multiplicando complejidad.[⁶]
- DB mal modelada: no pensar en entidades clave (cliente, proyecto, horas, facturas) desde el inicio, generando tablas ad hoc difíciles de escalar.[³⁶]
- Dependencia excesiva de herramientas de app-building (Lovable/Bolt) sin control sobre el código.[⁷⁴][¹³]

Errores de producto

- No hablar con usuarios internos (PMs, creativos, cuentas) antes de priorizar funcionalidades; el OS no refleja el trabajo real.[⁵³][²⁰]
- Foco en features "sexy" (AI avanzada, dashboards complejos) antes de resolver lo básico (claridad de tareas, pipeline, tiempo).[⁶³][³⁶]

Errores de UX

- Formularios pesados para operaciones recurrentes (logging de horas, creación de tareas); la gente deja de usar la herramienta.^[28]^[33]
- Falta de vistas por rol; todo el mundo ve lo mismo aunque sus necesidades sean distintas.^[^7]

Errores de desarrollo con IA

- Confianza ciega en IA sin revisión humana; bugs y deuda técnica oculta.^[68]^[67]
- No conservar un estilo de código consistente; cada módulo parece escrito por un "desarrollador distinto" (distintos modelos/prompts).^[^12]
- Consumo masivo de tokens en builders sin valor añadido claro.^[^74]

Cómo evitarlos

- Definir una **visión de producto** y un responsable claro (Product Owner interno).
- Empezar con monolito modular y buen diseño de datos; usar IA como asistente, no como dueño del diseño.^[65]^[12]
- Priorizar casos de uso diarios: registrar horas, ver qué hacer hoy, ver estado de proyectos y pipeline.
- Establecer **guidelines de uso de IA**: tipos de tareas permitidas, revisión obligatoria, límites de tokens por día.

Parte 9 — Roadmap recomendado (12 meses)

Fase 0: Investigación (Mes 0–1)

- **Objetivos**
 - Mapear procesos reales de la agencia: CRM, proyectos, tiempo, finanzas, documentación.^[3]^[20]
 - Identificar 3–5 "momentos de dolor" diarios (ej.: "no sé qué proyectos son rentables").
- **Entregables**
 - PRD del Agency OS (versión interna), mapa de procesos, diagrama de entidades.
- **Riesgos**
 - Sesgos de visión de dirección vs equipo; subestimar complejidad real.
- **Criterios para avanzar**
 - Aprobación de PRD y consenso en las prioridades del MVP.

Fase 1: Diseño (Mes 1–2)

- **Objetivos**
 - Diseñar arquitectura técnica y UX principales; decidir stack (Next.js, DB, auth, hosting).^{[14][6]}
- **Entregables**
 - Esquema DB, mapa de módulos, wireframes de pantallas clave, guía de diseño UI.
- **Riesgos**
 - Over-design; invertir demasiado tiempo en detalles estéticos antes de validar funcionalidad.
- **Criterios para avanzar**
 - Aceptación de diseño por stakeholders; alineación entre tech y negocio.

Fase 2: MVP (Mes 2–6)

- **Objetivos**
 - Construir e implementar CRM básico, proyectos/tareas, time tracking y wiki; integrar autenticación.^{[4][20]}
- **Entregables**
 - Versión usable del Agency OS para uso interno limitado, documentación técnica y de usuario.
- **Riesgos**
 - Feature creep, retrasos por intentar pulir demasiado.
- **Criterios para avanzar**
 - Equipo usando el sistema en al menos 50% de proyectos internos durante un mes.

Fase 3: Uso interno intensivo (Mes 6–9)

- **Objetivos**
 - Migrar gestión de clientes y proyectos al OS; medir utilización y margen; recoger feedback.^{[36][6]}
- **Entregables**
 - Informes internos de utilización/margen, backlog de mejoras.
- **Riesgos**
 - Resistencia al cambio; procesos no documentados que chocan con la herramienta.
- **Criterios para avanzar**
 - Uso consistente por el equipo; reducción visible de caos (por ejemplo, menos herramientas paralelas).

Fase 4: Optimización (Mes 9–11)

- **Objetivos**
 - Añadir capacidades de finanzas, planificación de recursos y dashboards; refinar UX y rendimiento.^{[4][33]}
- **Entregables**
 - Módulo de presupuestos/facturas, vista de carga por persona, dashboards clave.
- **Riesgos**
 - Sobre-automatizar sin gobernanza; crear flujos demasiado rígidos.
- **Criterios para avanzar**
 - Dirección y equipo perciben mejoras en decisiones (ej.: saber qué proyectos son rentables, cuándo contratar).

Fase 5: Posible SaaS (Mes 11–12+)

- **Objetivos**
 - Evaluar viabilidad de multi-tenant y oferta externa; definir propuesta de valor para otras agencias.^{[1][40]}
- **Entregables**
 - Diseño alto nivel de multi-tenant, pricing hipotético, landing page de producto.
- **Riesgos**
 - Distraerse del uso interno antes de consolidarlo; subestimar competencia.
- **Criterios para avanzar**
 - Decisión explícita de explorar SaaS (pilotos con 1–3 agencias amigas) o mantenerlo como OS interno.

RECOMENDACIÓN EJECUTIVA

Para los próximos 12 meses, la estrategia más inteligente para construir este Agency OS es:

1. **Tratarlo primero como sistema operativo interno, no como producto.** Copiar los conceptos clave de Productive/Teamwork/Notion Agency OS (CRM, proyectos, tiempo, rentabilidad, documentación), pero reducir el alcance al mínimo que la agencia use cada día.^{[5][20][^4]}
2. **Elegir una arquitectura simple y estándar:** Next.js en Vercel con Postgres+Prisma en Supabase/Railway, Clerk/Auth.js para auth y Metabase como BI opcional. Esto reduce costes, acelera desarrollo y es familiar para cualquier SaaS moderno.^{[10][14][^6]}
3. **Construir un monolito modular bien diseñado,** evitando microservicios hasta que haya señales claras de necesidad. Estructurar el dominio en torno a Cliente, Proyectos, Horas y Facturas, de forma que luego sea fácil añadir multi-tenant o exponer APIs.^{[10][36]}
4. **Aprovechar IA de forma pragmática:** usar Cursor o Windsurf + Claude Sonnet/Opus para diseño, scaffolding y refactorizaciones, y v0 para generar UI, manteniendo control humano

sobre arquitectura y estilo de código.^{[67][68][^12]}

5. **Empezar con un MVP muy enfocado (Fases 0–2)** que entregue valor interno claro en menos de 6 meses: CRM mínimo, proyectos/tareas, time tracking y wiki de SOPs vinculados a proyectos. Medir adopción antes de ampliar.^{[20][6]}
6. **Usar el OS como laboratorio** para perfeccionar procesos de agencia (SOPs, reporting, planificación de recursos) durante meses, hasta que la operación interna sea notablemente más eficiente.^{[53][36]}
7. **Solo entonces evaluar la transición a SaaS:** identificar nichos (agencias pequeñas que no quieren Productive/Scoro), validar con 1–3 clientes piloto y diseñar multi-tenant sin romper la simplicidad inicial.^{[41][1]}

Este enfoque equilibra velocidad de desarrollo, simplicidad y capacidad futura de crecer: evita inversiones prematuras en complejidad enterprise, mientras construye un asset valioso que mejora la operación diaria y abre la puerta a un producto comercial en el momento adecuado.

References

1. [10 Best Agency Management Software Platforms in 2026](#) - 1. Nymble, best overall for growing agencies · 2. Productive, best for mid-size agencies focused on ...
2. [Building the Ultimate Marketing Tech Stack for 2025](#) - With 8,000+ marketing tools available today, the challenge isn't finding tools—it's figuring out whi...
3. [Building the Perfect Marketing Technology Stack for Your Agency](#) - Learn how to build the perfect marketing technology stack for your agency with strategic integration...
4. [Agency Management & Professional Services Automation ...](#) - Run resources, projects, and finances in one platform. Productive helps agencies and consultancies s...
5. [#1 Agency Project Management Software | Teamwork.com](#) - Run your agency more efficiently with professional services automation software for resource plannin...
6. [The Best Agency Tech Stack in 2026: PM, Time Tracking, Files ...](#) - The agency tech stacks that actually work at 5, 15, and 50 people in 2026 — with integration gotchas...
7. [Notion vs Asana vs ClickUp vs Monday \(2025\)](#) - LaunchAdvisor - Choosing the right project management tool? We compare Notion, Asana, ClickUp, and Monday side by si...
8. [How to Build an Interactive Dashboard in Retool \(Tutorial\)](#) - Learn how to build a Retool dashboard with AI-assisted development, connected data, KPI cards, chart...
9. [Retool | Build internal software better, with AI.](#) - Build, deploy, and manage internal tools with Retool's unified engine. Connect to any database, API,...
10. [The Data Stack Report 2025](#) - Explore real-world trends in data tooling with the 2025 Metabase Community Data Stack Report. Based ...
11. [Bolt vs Lovable vs v0: which AI app builder ships fastest in 2026](#) - Bolt vs Lovable vs v0 compared honestly for non-developers in 2026. Pricing, output, deployment, loc...
12. [Claude Code vs Cursor vs Windsurf: Comparativa ...](#) - Claude Code es mejor para refactorizaciones masivas y proyectos grandes, Cursor para desarrollo diar...

13. [Bolt vs Lovable vs V0: Which One to Choose in 2026?](#) - Lovable has a more no-code/low-code approach than Bolt. Users describe their app in plain language, ...
14. [Top 10 Tools Every Modern Agency Needs to Scale in 2026](#) - Agencies in 2026 face a familiar paradox: clients expect faster delivery, higher quality, and broad...
15. [Top 16 marketing agency tools in 2026 - Funnel](#) - Learn about the best marketing agency tools to add to your stack in 2026, including how to build the...
16. [The best agency management software in 2026 - Teamwork.com](#) - Compare the 10 best agency management software tools for 2026. See features, pricing, and which plat...
17. [12 best agency management software tools I tested in 2026](#) - If you want faster implementation or simpler interfaces for smaller teams, Teamwork or Productive mi...
18. [6 Scoro Alternatives for Agencies in 2026 - Workamajig](#) - We review the 6 best Scoro alternatives for agencies, including Workamajig, Advantage, Teamwork, Pro...
19. [The Ultimate Notion Agency Dashboard \(AGENCY OS 2025 Walkthrough\)](#) - Replace messy spreadsheets & expensive agency software with Notion: Manage all areas of your agency ...
20. [The 5 Notion Systems Every Scaling Agency Needs](#) - CRM, delivery OS, client portal, SOPs, capacity planning — the five connected Notion systems scaling...
21. [Notion for creative agencies: managing internal and client work](#) - Notion gives your team a central hub to organize all your internal and client-facing work, from team...
22. [Pipedrive vs. HubSpot vs. Salesforce: Which CRM Reigns Supreme ...](#) - A founder-friendly breakdown of the top CRMs—what they do best, who should use them, and how to choo...
23. [HubSpot vs. Pipedrive | Sales Software Comparison Guide](#) - HubSpot Sales Hub drives 86% better lead quality for customers. Compare features & pricing for these...
24. [HubSpot vs Salesforce vs Pipedrive: Best CRM 2025](#) - We used all three CRMs for real sales pipelines. Here's the honest breakdown of pricing, features, a...
25. [Salesforce vs HubSpot vs Pipedrive: CRM Showdown - Barawave](#) - Introduction – Why This CRM Showdown Matters In the world of sales, marketing, and customer relation...
26. [Best CRM for 2026: Pipedrive vs HubSpot vs Salesforce | Sales House](#) - Which CRM should you actually use in 2026? Real comparison of Pipedrive, HubSpot, and Salesforce bas...
27. [HubSpot vs. Salesforce vs. Pipedrive: 3 CRMs im Vergleich](#) - HubSpot, Salesforce und Pipedrive sind drei beliebte CRM-Tools, doch welches eignet sich für Ihr Unt...
28. [Clickup vs Monday vs Notion vs Asana | \(2025\) Which Is The ULTIMATE Automation Tool To Use Right Now](#) - ClickUp vs Monday vs Notion vs Asana | (2025) Which Is The ULTIMATE Automation Tool To Use Right Now...
29. [Notion vs ClickUp, Monday, and Airtable: a fair look - Raccoon Page](#) - Notion versus the workspace tools it's most often compared to — ClickUp, Monday, and Airtable. What'...
30. [What Is The Best Project Management Software? \(Asana vs ClickUp vs Notion vs Airtable vs Monday.com\)](#) - I'm breaking down and ranking the top project management platforms in this ultimate tier list. Wheth...
31. [Notion Masterclass: Build an OS for Your Digital Agency \(2025\)](#) - Here's how to manage your Digital Agency & Business from one single place, and stop switching between...

32. [\[PDF\] How to Build a Winning Marketing Tech Stack in 2025 \(Without ...](#)
33. [Teamwork Review Ireland 2026 - Project Management ...](#) - Teamwork is a project management platform built in Cork, Ireland. Here's what it offers, how it pric...
34. [Product Tour: Features Made To Manage Client Work - Teamwork.com](#) - Explore the professional services automation features that help teams manage projects, resources, bu...
35. [Teamwork Review 2026 — Still a Top Choice or Time to Move ...](#) - Get an honest look at Teamwork in this review. See if it's still a top choice for project management...
36. [End-to-End Agency Management Guide - Productive.io](#) - How profitable is your agency? Learn how to get the most out of your agency management system so you...
37. [Productive Reviews 2026: Details, Pricing, & Features | G2](#)
38. [Productive Reviews & Features 2026 - OMR](#) - Are you interested in Productive? On OMR Reviews you can find out what other users think about Produu...
39. [July Update: Standardized Billable Time](#) - July 2025. You can create internal projects to separate them from client work. We've also standardiz...
40. [Scoro vs. Productive: Comparativa y Reseñas Expertas para 2026](#) - Compara Scoro y Productive para la gestión de proyectos. Descubre qué herramienta ofrece las mejores...
41. [12 Best Scoro Alternatives in 2026 \(Ranked by Cost\)](#) - Compare 12 Scoro alternatives with real pricing from free to \$50/user. Annual cost tables for 10 and...
42. [Scoro: Complete Agency...](#) - The Pattern Cloud | Find the right project management software for your creative agency. Expert-test...
43. [Who has experience with PM tools like Teamwork, Hellobonsai or ...](#) - I'm trying to pick software to help manage my social media services agency, and I've found a few tha...
44. [\[8 Best CRM and Invoicing Software for Creative Agencies 2025 ...](#) - Run your entire agency from one place. One place for managing clients, billing, onboarding, projects...
45. [Retool Review - Should You Use It For Internal Tools? \(2025\)](#) - Retool Review - Should You Use It For Internal Tools? (2025)

Comprehensive review of Retool low-cod...

46. [Retool Review 2025: The BEST Internal Tool Builder? 🏆 \(Real Use Cases\)](#) - retool, retool review, retool tutorial, retool vs appsmith, retool vs budibase, retool vs supabase, ...
47. [2025 Metabase Data Stack Report: Trends and Challenges](#) - The 2025 Metabase Data Stack Report is here. This year's report goes beyond the hype to reveal how r...
48. [The 2026 Creative Operations Stack: 6 Tools to Scale Production](#) - Scale creative production without increasing headcount. Discover the 6-tool Creative Operations Stac...
49. [Marketing Tech Stack 2025: Tools Every Digital Agency ...](#) - Build the ultimate marketing tech stack for 2025. Discover the essential CRM, AI, analytics, and aut...
50. [Notion Agency OS: Template Walkthrough](#) - Create a Free Notion Account:
<https://affiliate.notion.so/d9s87wx622u5>

Agency OS: <https://simonotio...>

51. [Modern Marketing Data Stack Report 2026 5th Edition | Snowflake](#) - Explore how AI agents are moving from experiment to execution, with governance a key factor for scal...
52. [Best Agency Project Management Software • November 2025](#) - Find the best Agency Project Management software of 2025. Get discounts on top-rated systems and too...
53. [Five Marketing Shifts That High-Performing Agencies Are Acting On](#) - Research finds that agencies are redefining success with brand investment, AI adoption, advanced mea...
54. [Agency Tool Stack 2026: The Definitive Guide by Function + ...](#) - How to choose and implement the right agency stack across PM, CRM, proposal software, analytics, and...
55. [Building the Right Tech Stack for a Marketing Agency](#) - Find how to design a lean, growth ready marketing tech stack for your agency with core tools and sel...
56. [Digital Agency Operating Stack Guide: CRM, Client Reporting, SEO, Delivery, Profitability, Data, PPC, Communication, Design, Creative, and Social Tools for 2026](#) - Choose a digital agency tool stack for 2026 across CRM, client reporting, SEO, delivery, profitabili...
57. [Top 5 Tools for Creative Operations in 2026 - Medialake AI](#) - Creative Operations is undergoing one of the most significant transformations in its history. What w...
58. [The Internal Tools Teams Are Replacing With Retool](#) - See which types of internal tools teams replace or consolidate with Retool as they reduce software s...
59. [12 Best All-in-One Software for Agencies \[2026\] - Agiled](#) - Compare 12 all-in-one agency platforms by price, minimum seats, retainer billing, and stack consolid...
60. [The Future of Digital Marketing: Trends to Know in 2025 - StackAdapt](#) - Find out what's shaping digital marketing in 2025 and beyond—from the convergence of adtech and mart...
61. [The agency software stack in 2026, ranked by layer - Phloz](#) - The 2026 agency stack by layer — what each layer does, the top picks, the tools to drop, and the con...
62. [Salesforce vs HubSpot vs Pipedrive: CRM Comparison for Sales ...](#) - Salesforce, HubSpot, or Pipedrive? This no-fluff comparison covers pricing, AI, pipeline management,...
63. [12 Best AI Tools for Agency Workflows in 2026 \(Save 10+ Hours Per ...](#) - We tested 12 AI tools agencies are actually using in 2026. See which save the most time on client wo...
64. [Enterprise CRM UI/UX Design Patterns: A Guide for CTOs](#) - Discover the great examples of enterprise applications CRM UI UX design patterns. Learn best practic...
65. [La guerra de los IDEs AI en 2026: Cursor, Windsurf y Claude Code](#) - La guerra de los IDEs AI en 2026: Cursor, Windsurf y Claude Code — ¿quién gana? ✕ Hace un año, la co...
66. [Best Vibe Coding Tools in 2026 — Ranked, Rated & Compared](#) - Discover the best vibe coding and AI development tools in 2026. This comparison analyzes 34 leading ...
67. [12: Vibe Coding Revolution: Top 10 AI Tools Transforming Development in 2026](#) - Software development is entering a completely new era — one driven by AI agents, natural language wo...
68. [Cursor vs Windsurf vs Claude Code in 2026 - DEV Community](#) - A deep, no-BS comparison of the three dominant AI coding tools in 2026. We tested Cursor, Windsurf, ...
69. [Cursor vs Windsurf vs Claude Code: Best AI Coding Tool in 2026 \(Now ...](#) - Cursor vs Windsurf vs Claude Code compared side-by-side. Pricing (\$15-\$20/mo), benchmarks, features,...

70. Lovable vs Bolt.new vs v0: Best AI App Builder in 2026 - Particula Tech - We built the same MVP on all three platforms. Lovable ships full-stack fastest, Bolt.new gives devel...
71. v0 vs Bolt vs Lovable: Best AI App Builders & Vibe Coding Tools ... - v0, Bolt, and Lovable compared for building apps with AI. See pricing, code quality, deployment opti...
72. Mejores Herramientas de Vibe Coding 2026 - Javadex - Las 8 mejores herramientas de vibe coding en 2026 analizadas: Cursor AI, v0.dev, Bolt.new, Claude Co...
73. Best Vibe Coding Tools in 2026 (Honest Ranking) - Autonoma AI - Best vibe coding tools in 2026 compared: Cursor, Windsurf, Bolt.new, Claude Code and more. Honest ra...
74. Bolt Vs Lovable Vs V0: Which AI App Builder Actually Works? (2026) - ☒ Check Current Price on Amazon: <https://amzn.to/3l8udfq>
☒ Bookmark & Use for ANY Amazon Purchase (...)
75. V0 vs Bolt.new vs Lovable: Best AI App Builder 2026 ... - Complete 2026 comparison of V0 vs Bolt.new vs Lovable AI app builders. Detailed analysis of pricing,...
76. AI Coding Tools (Cursor, Claude Code, Lovable, Replit) vs ... - AI coding tools vs hiring a software team in 2026. When Cursor, Claude Code, Lovable and Replit win,...
77. AI builder comparisons - Lovable vs v0 vs Cursor vs Bolt ... - Transform your business ideas into comprehensive, AI-ready Product Requirements Documents. Generate ...
78. The Modern Agency Tech Stack 2026: CRM, PM & Client Portal Guide - Build the right agency tech stack in 2026. Honest comparison of CRM, project management, and client ...